Bst Construction

AlgoExpert · Medium · Binary Search Trees

The problem

- A BST class supporting insert, contains and remove (duplicates go right; removing a node with two children replaces it with its in-order successor's value).
- Removing the only node is a no-op.

Intuition

- Insert, search and remove all follow the BST ordering; removal of a two-child node swaps in the smallest value of the right subtree.

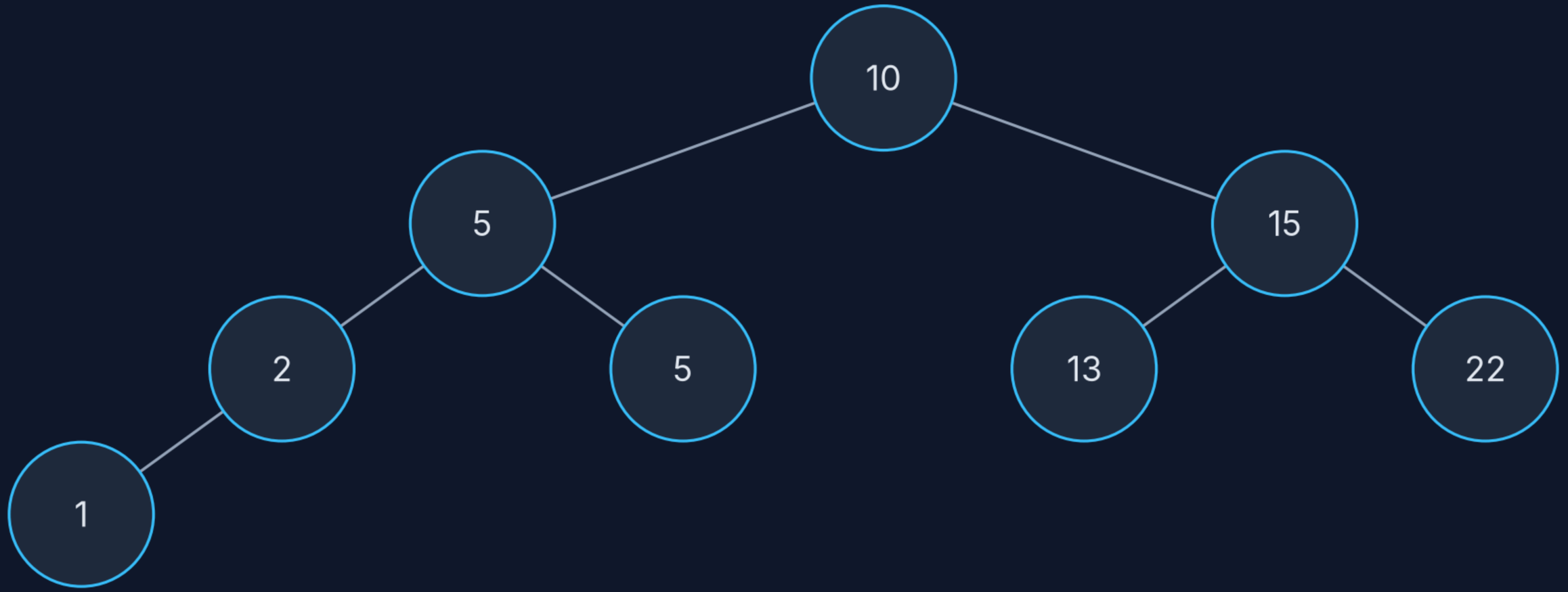
Brute force vs optimal

- Unbalanced BST ops: O(h) — O(log n) average, O(n) worst
- Alternative: sorted array → O(log n) search but O(n) insert
- Self-balancing trees (AVL, red-black) guarantee O(log n)

Approach

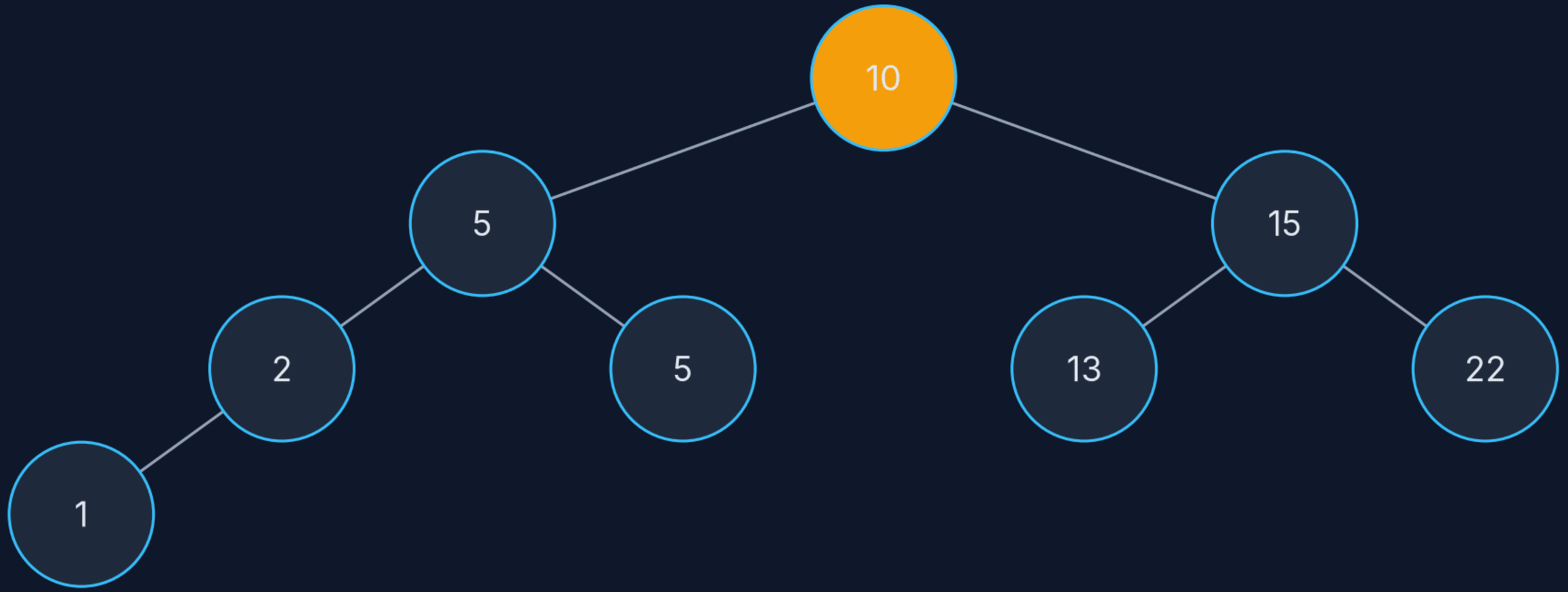
- 1. Walk left or right by comparison
- 2. Insert at the empty spot
- 3. To remove with two children, copy the right subtree's minimum and delete it there
- 4. Handle the root carefully

Worked example



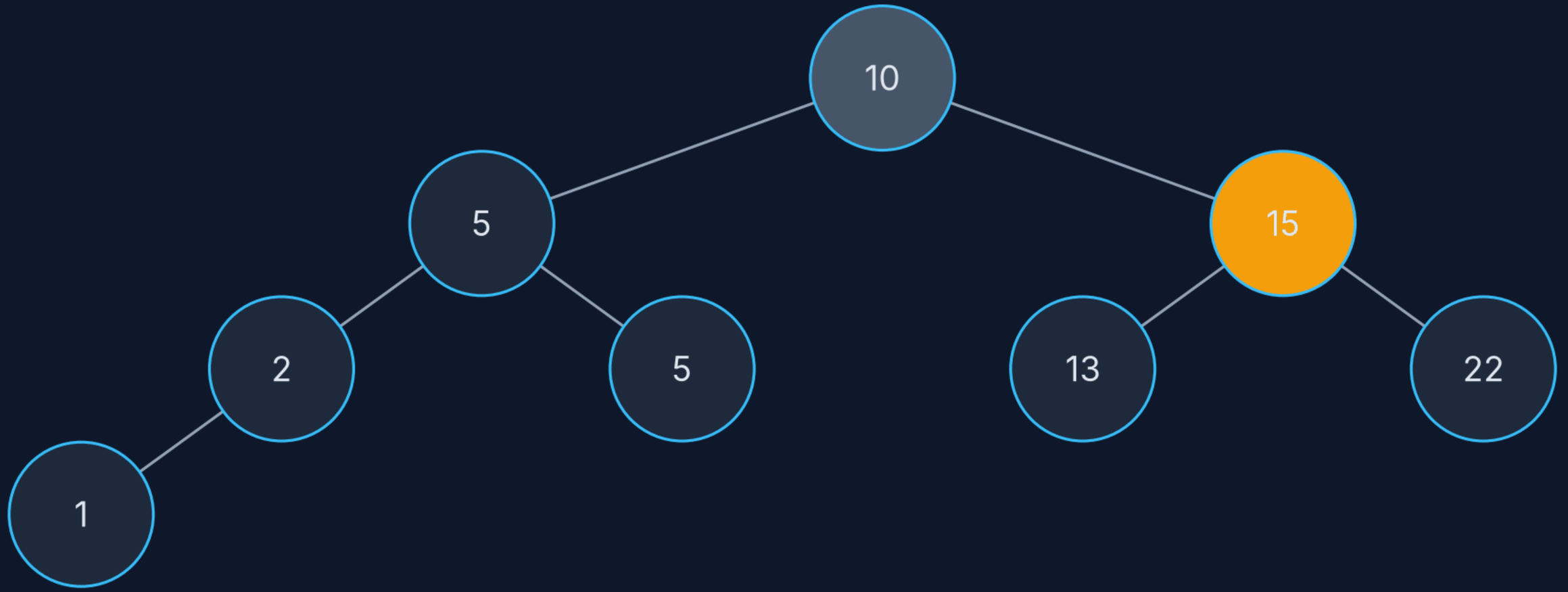
Start: BST built from 10, 5, 15, 2, 5, 13, 22, 1

Worked example



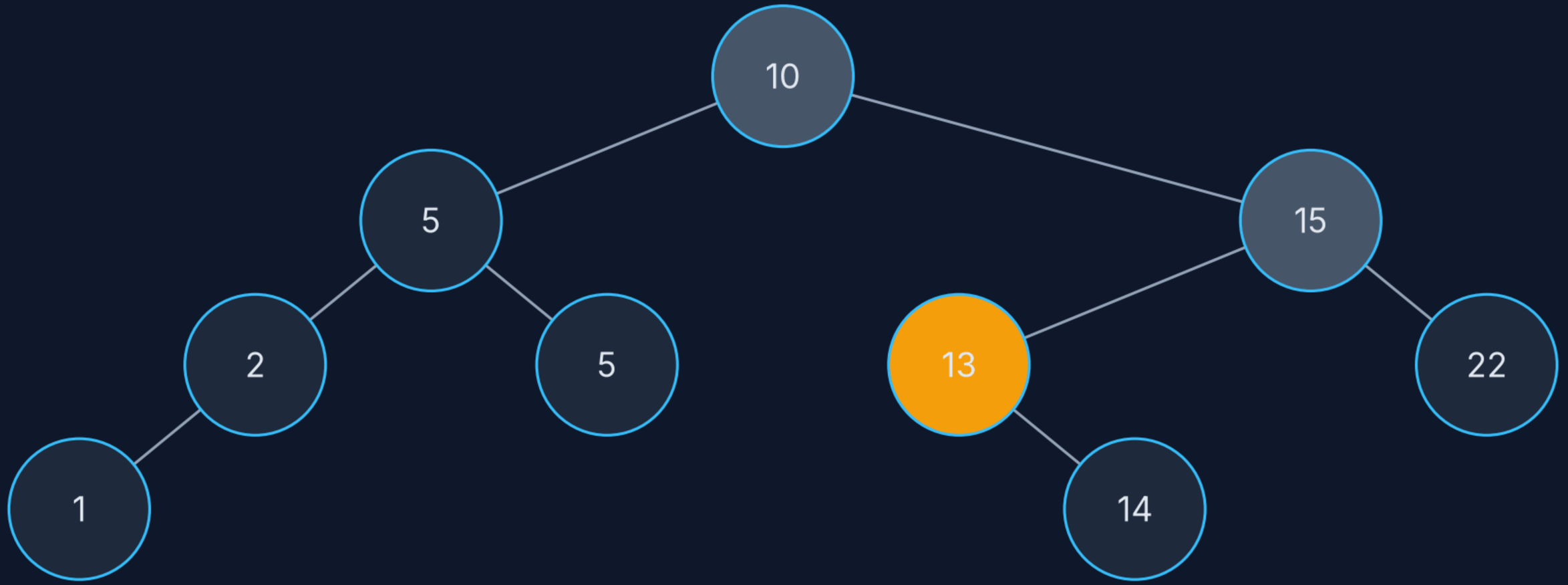
insert(14): at 10 → go right

Worked example



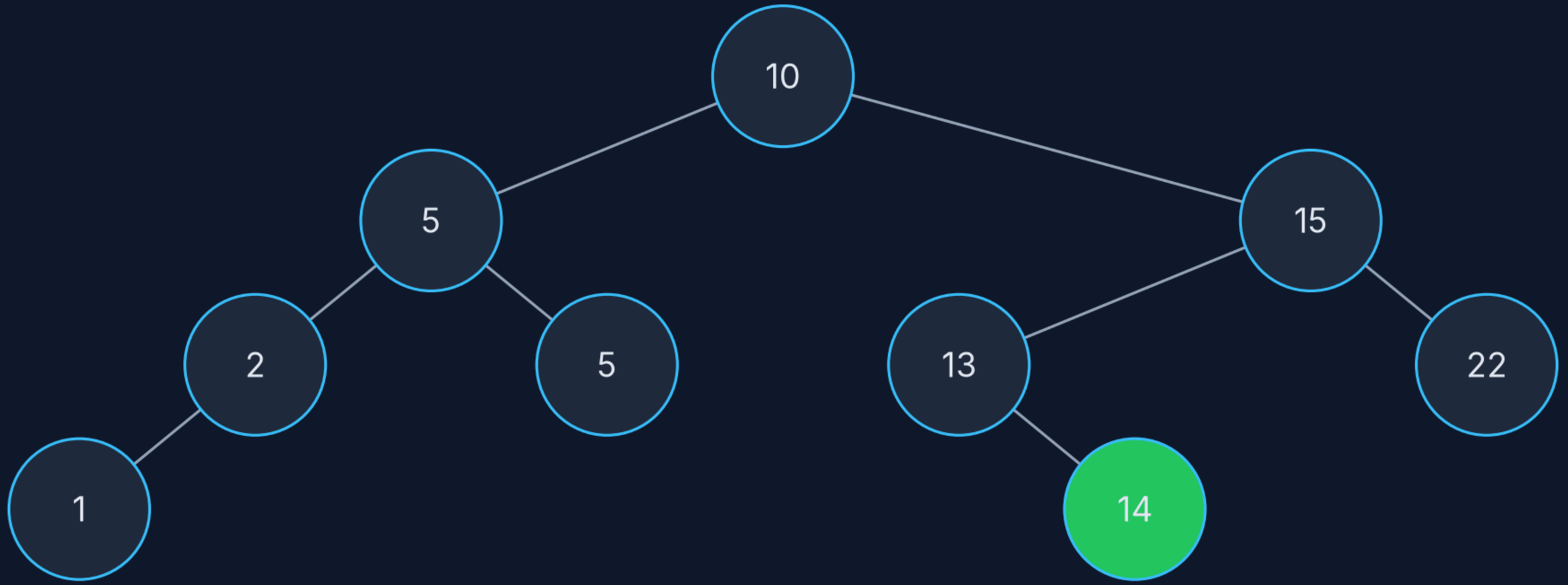
insert(14): at 15 → go left

Worked example



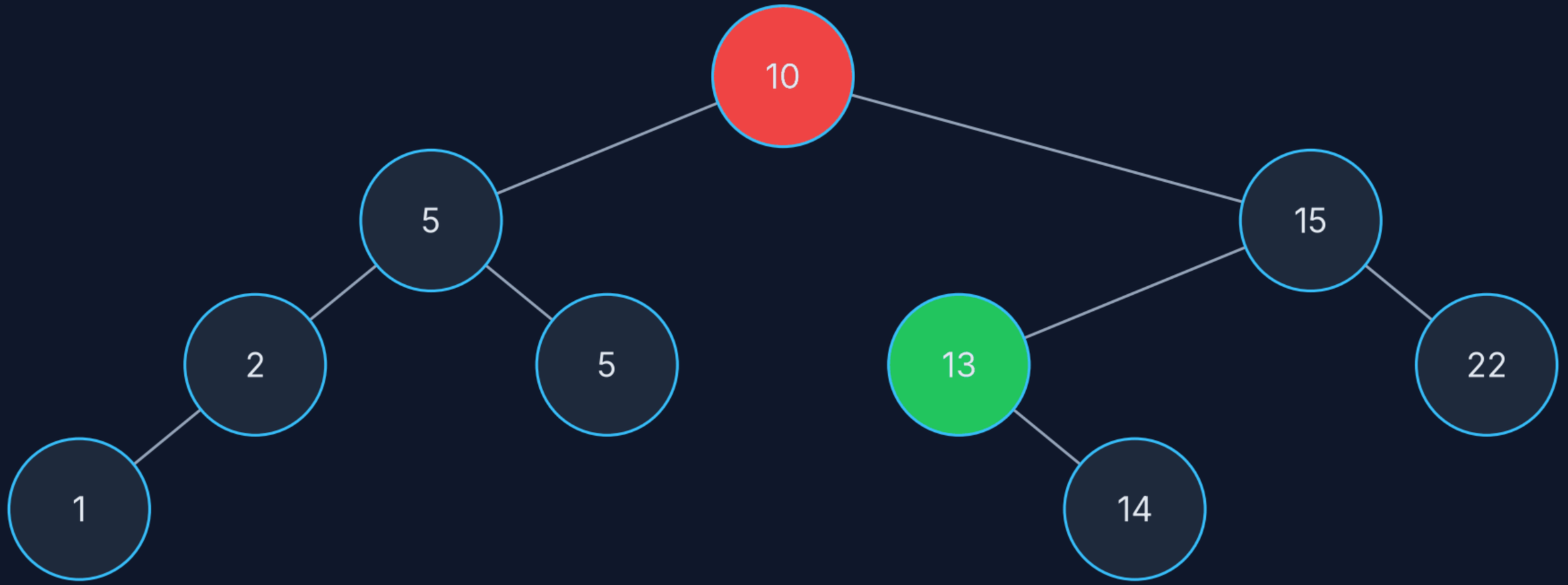
insert(14): at 13 → go right

Worked example



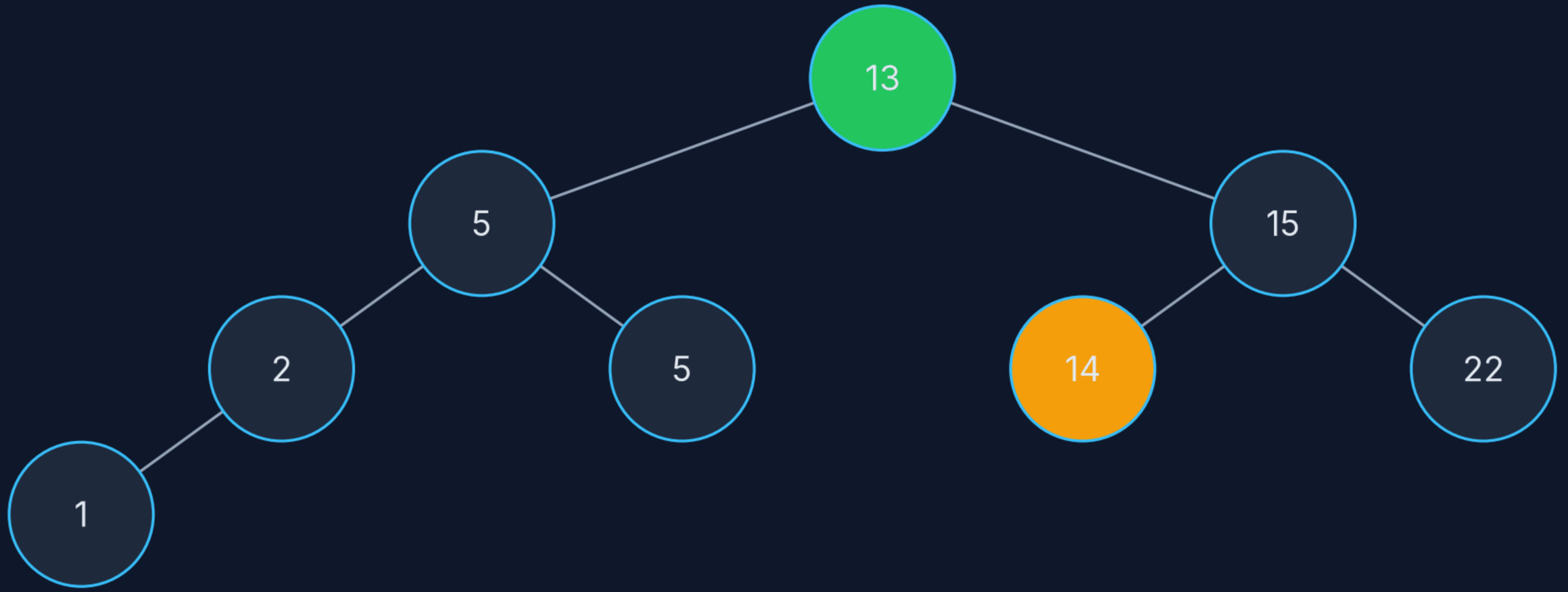
14 becomes 13's right child

Worked example



remove(10): two children → copy min of right subtree (13)

Worked example



Result: 13 at the root; 14 moves up

Java solution (1/2)

```java
public class BstConstruction {
    public static class BST {
        public int value;
        public BST left, right;

        public BST(int value) { this.value = value; }

        public BST insert(int v) {
            BST cur = this;
            while (true) {
                if (v < cur.value) { if (cur.left == null) { cur.left = new BST(v); return this; } cur = cur.left; }
                else { if (cur.right == null) { cur.right = new BST(v); return this; } cur = cur.right; }
            }
        }

        public boolean contains(int v) {
            BST cur = this;
            while (cur != null) {
                if (v == cur.value) return true;
                cur = v < cur.value ? cur.left : cur.right;
            }
            return false;
```

Java solution (2/2)

```
        }

        public BST remove(int v) { remove(v, null); return this; }

        private void remove(int v, BST parent) {
            BST cur = this;
            while (cur != null) {
                if (v < cur.value) { parent = cur; cur = cur.left; }
                else if (v > cur.value) { parent = cur; cur = cur.right; }
                else {
                    if (cur.left != null && cur.right != null) {
                        cur.value = cur.right.minValue();
                        cur.right.remove(cur.value, cur);
                    } else if (parent == null) {
                        BST child = cur.left != null ? cur.left : cur.right;
                        if (child != null) { cur.value = child.value; cur.left = child.left; cur.right = child.right; }
                    } else if (parent.left == cur) parent.left = cur.left != null ? cur.left : cur.right;
                    else parent.right = cur.left != null ? cur.left : cur.right;
                    return;
                }
            }
        }

        private int minValue() { BST c = this; while (c.left != null) c = c.left; return c.value; }
    }
```

Complexity

- Time: O(log n) avg, O(n) worst per op
- Space: O(1) (iterative)

Edge cases & pitfalls

- Remove the root when it has one child (no parent to relink)
- Equal values go right — be consistent
- Remove a value that doesn't exist → no-op

Interview tips & follow-ups

- Iterative versions use O(1) space
- Follow-up: make it self-balancing; add size for rank queries

Recap

- Insert, search and remove all follow the BST ordering; removal of a two-child node swaps in the smallest value of the right subtree.
- Complexity: O(log n) avg, O(n) worst per op